{JS}
MASTERY

# Complete Claude Code Course Flow

The important thing is that the project now grows **with the course**.

# Foundations

## 01 Start the Project

Create an empty folder and launch Claude Code inside it.

BASH

```
mkdir gucci-store
cd gucci-store
claude
```

Then give Claude a small initial setup task.

PROMPT

Set up a new Next.js ecommerce application in this folder using TypeScript and Tailwind CSS.

Create the initial project structure and install the basic dependencies we need to start building.

Keep the application minimal for now.

Do not build authentication, database schemas, payments, admin features or the entire ecommerce application yet.

The point is to start with something Claude can actually understand and evolve instead of generating the whole project at once.

### 02 Explore the Project

Before asking Claude to change anything else, make it inspect what currently exists.

PROMPT

Explore this project and explain the current architecture, technologies, important folders and how the main pieces connect.

Watch how Claude uses tools such as `Glob`, `Grep` and `Read` to understand the repository.

---

LESSON 02

## CLAUDE.md & Project Memory

### 03 Create Project Context

Run:

```
/init
```

Claude will inspect the repository and generate a starting `CLAUDE.md`.

Then ask it to clean that file up (if needed).

PROMPT

Review the current CLAUDE.md against the actual project.

Keep only instructions Claude genuinely needs to remember across future sessions.

Remove anything that is obvious from reading the repository. Don't add generic instructions like "write clean code" or "follow best practices."

You want `CLAUDE.md` to contain durable project context, not a copy of your README.

# Permissions & Building the UI

## 04 Study the Gucci Design Language

Now you can start building the actual application.

Before creating pages, have Claude study the design reference and turn what it learns into real project conventions.

PROMPT

Study the current Gucci website as a visual reference for this ecommerce application.

Analyze its typography, spacing, layout, navigation, product presentation, imagery, buttons, borders, colors and responsive behavior without copying its text, assets or branding.

Then implement those principles in our project by setting up the global design system, including typography, color tokens, spacing, containers, buttons, links, borders and reusable responsive primitives in the appropriate Tailwind and CSS files.

Briefly explain the rules you implemented and the files you changed.

The important part is that Claude should **implement** the design system, not just describe Gucci's website.

## 05 Build the Homepage

Now build the first complete customer-facing feature.

PROMPT

Build the homepage for the ecommerce application using the design system already established in the project.

Take inspiration from premium fashion ecommerce experiences with large editorial imagery, featured collections and product-focused sections.

Use simple sample product data where needed and use images from unsplash and other sources. Make the page responsive and verify the result when you're done.

Don't move on immediately.

Review the result and iterate on anything that doesn't feel right.

---

## 06 Refine the Homepage

If the result feels too generic:

PROMPT

Review the current homepage and improve anything that feels generic, inconsistent or too template-like.

Keep the existing design system.

Focus on typography, layout, hierarchy, spacing, product presentation and responsive behavior. Don't redesign unrelated parts of the application.

## 07 Build Product Listing

Continue building the storefront one feature at a time.

PROMPT

Build the product listing experience using the existing design system.

Use sample fashion products and categories for now.

Keep filters and controls visually restrained and make the product imagery the focus. Reuse existing components where appropriate and make the page responsive.

At this stage, sample data is intentional. You haven't reached the database lesson yet.

---

## 08 Build Product Details

PROMPT

Build the product detail experience.

Include large product imagery, product information, price, category and a simple stock state.

Keep the page consistent with the homepage and product listing. Reuse the design system and existing components wherever possible.

You now have enough of the storefront built that replacing sample data with a real database actually makes sense.

# Plan Mode & Database

## 09 Plan the Database

Enter Plan Mode:

```
/plan
```

Then:

PROMPT

Inspect the current ecommerce project and plan how to replace the sample product data with PostgreSQL using Drizzle ORM.

For this first version, we only need:

- products
- categories
- stock

Study how the existing storefront currently uses product data before deciding the schema.

Keep the database simple.

Do not add carts, orders, payments, wishlists, reviews, warehouses or product variants yet.

Show me:

- the proposed schema
- relationships
- how price should be stored
- how stock should be represented
- migrations
- files that need to change
- how the existing storefront will consume the database
- how we'll verify the implementation

This is where Plan Mode is actually useful because a wrong schema becomes expensive once the rest of the application depends on it.

## 10 Simplify an Over-Engineered Plan

If Claude starts designing systems you don't need:

PROMPT

This is more complex than the current application needs.

Keep inventory simple and store the current stock directly on the product.

Remove warehouses, inventory locations, reservations, inventory movements and anything else that isn't needed for the application we're currently building.

---

## 11 Save the Database Conventions

Once you agree with the plan:

PROMPT

Update CLAUDE.md with database conventions from the plan we approved. Keep it short. Don't add information Claude can easily discover from the codebase.

## 12 Configure PostgreSQL

Create the database yourself and add `DATABASE_URL` to `.env.local`.

Then tell Claude:

PROMPT

The PostgreSQL database is ready and DATABASE_URL is available in .env.local.

Use process.env.DATABASE_URL where needed without trying to read the environment file.

Continue with the approved database plan.

---

## 13 Implement the Database Plan

PROMPT

Implement the database plan we approved.

Set up Drizzle, the database connection, products, categories, stock, migrations and development seed data.

Then replace the current sample product data in the storefront with real database queries.

Keep the implementation limited to the approved plan.

Do not add unrelated ecommerce features. Run the relevant checks and verify the storefront still works when you're done.

## 14 Verify Against the Plan

PROMPT

Review the database implementation against the plan we approved.

Check:

- what was implemented correctly
- anything implemented differently
- anything still missing
- any reproducible problems

Don't expand the scope. Fix only issues required to make the approved plan complete.

Then inspect the diff yourself:

```
git diff
```

LESSON 05

# Skills & Reusable Workflows

By this point, you've repeatedly told Claude to inspect the existing UI, reuse the design system, keep responsive behavior consistent and verify the result.

That's exactly when a Skill becomes useful.

## 15 Create the Build UI Skill

Create:

```
.claude/skills/build-ui/SKILL.md
```

with:

SKILL.MD

```
---
description: Build or update customer-facing ecommerce UI using the existing design
system, components, product patterns and responsive conventions. Use when creating
or significantly changing customer-facing pages, sections or components.
---

Build or update the following UI:

$ARGUMENTS

Follow this process:

1. Inspect the current implementation and related components first.
2. Search for reusable components before creating new ones.
3. Reuse the existing design system and patterns.
4. Keep the change focused on the requested feature.
5. Keep responsive behavior consistent.
6. Follow the current product and database patterns.
7. Run and visually verify the application.
8. Run the normal project checks.
9. Fix reproducible issues without expanding scope.
10. Summarize what changed.
```

From this point onward, use `/build-ui` whenever you're building suitable customer-facing UI.

You no longer need to repeat the same instructions in every prompt.

---

## 16 Build New Arrivals With the Skill

PROMPT

`/build-ui` Build a New Arrivals page using the newest products from the database.

The Skill already knows to inspect the project, reuse components, follow the design system and verify the result.

---

## 17 Build Category Pages With the Skill

PROMPT

`/build-ui` Build a category collection page using the existing categories and products from the database.

---

## 18 Build Product Search With the Skill

PROMPT

`/build-ui` Build a product search experience using the current product data, product cards and design system.

Keep the search focused on products.

# Authentication

Authentication now becomes the next meaningful application capability.

And because you've already learned Skills, this is also a good time to show that Skills don't only have to be ones you create yourself.

## 19 Add the Better Auth Skill

Better Auth provides an Agent Skill that can give Claude focused guidance for implementing Better Auth correctly.

Install it:

BASH

```
npx skills add better-auth/skills
```

Then use Plan Mode before touching your database and route protection:

```
/plan
```

PROMPT

Inspect the current application and plan authentication using Better Auth.

We need:

- sign up
- sign in
- sign out
- persistent sessions
- protected customer account routes
- an admin role for protected admin routes

Use our existing PostgreSQL and Drizzle setup.

Use the installed Better Auth Skill when relevant.

Keep this first version focused.

Don't add social login, password reset, two-factor authentication or unrelated account features yet.

Show me:

- required database changes
- Better Auth configuration
- server-side session handling
- customer route protection
- admin authorization
- how we'll verify authenticated and unauthenticated flows

## 20 Implement Authentication

Once the plan looks correct:

PROMPT

Implement the approved Better Auth plan.

Use the installed Better Auth Skill where relevant. Add sign up, sign in, sign out and persistent sessions. Use the existing PostgreSQL and Drizzle setup.

Protect customer account routes for authenticated users. Protect admin routes and actions using the admin role server-side.

Do not add unrelated authentication features. Verify authenticated, unauthenticated and unauthorized flows when you're done.

## 21 Build Auth UI With `/build-ui`

You've already created the UI Skill, so use it.

PROMPT

`/build-ui` Build the sign-in and sign-up experiences. Include useful validation, loading and error states.

You don't need to repeat the whole design-system instruction anymore.

## 22 Build the Customer Account With `/build-ui`

PROMPT

`/build-ui` Build the authenticated customer account experience. Start with account information and navigation. Don't add order history yet.

SECTION

# CART

## 23 Plan the Shopping Cart

Before implementing the cart:

```
/plan
```

PROMPT

Inspect the current products, stock model and authentication setup and plan the shopping cart.

The cart needs to support:

- adding products
- changing quantity
- removing products
- current product pricing
- subtotal calculation
- preventing quantities above available stock

Choose the simplest appropriate persistence model for our current application.

Don't add Stripe or checkout yet.

Show me:

- cart data flow
- files involved
- stock validation
- server-side responsibilities
- how we'll verify the cart

## 24 Implement Cart Logic

PROMPT

Implement the approved shopping cart plan.

Support:

- adding products
- changing quantity
- removing products
- subtotal calculation
- stock validation

Do not trust arbitrary product prices supplied by the client. Use the current product data from the application.

Don't add payment functionality yet. Verify the cart behavior when you're done.

---

## 25 Build Cart UI With `/build-ui`

PROMPT

`/build-ui` Build the shopping cart interface around the cart functionality we just implemented. Include quantity controls, remove actions, subtotal, empty state and stock-related states.

SECTION

# STRIPE & CHECKOUT

Payments are another high-impact area, so use Plan Mode again.

## 26 Add Stripe's Claude Code Plugin

At the point where Stripe becomes relevant, install the official Stripe Claude Code plugin:

```
claude plugin install stripe@claude-plugins-official
```

Then enter:

```
/plan
```

PROMPT

Inspect the current products, cart, inventory and authentication flow and plan Stripe checkout. Use the installed Stripe plugin where useful.

Decide:

- which product and pricing data must come from our database
- what Stripe should own
- how orders should be represented
- when stock should be revalidated
- when orders should be created
- how payment success should be confirmed
- how Stripe webhooks should update orders
- how duplicate webhook events should be handled
- what should happen when payment fails or is abandoned

Do not trust client-provided prices or payment status.

Show me the database changes, payment flow, webhook flow and verification approach.

## 27 Add Orders to the Database

Once the architecture is approved:

PROMPT

Implement only the database changes from the approved checkout plan.

Add the order and order item structures needed for checkout. Associate orders with the authenticated customer.

Keep payment status and Stripe identifiers explicit. Use migrations for all schema changes.

Do not implement Stripe checkout yet. Verify the database changes before continuing.

---

## 28 Implement Stripe Checkout

PROMPT

Implement the approved Stripe checkout flow.

Use trusted server-side product data from PostgreSQL when calculating prices and totals.

Revalidate stock before beginning checkout. Create the required order records and initiate Stripe checkout. Handle checkout success and cancellation.

Do not mark an order as paid simply because the browser reaches the success page. Use the installed Stripe plugin where useful.

Run the relevant checks when you're done.

## 29 Implement the Stripe Webhook

PROMPT

Implement the Stripe webhook from the approved checkout plan.

Verify webhook signatures before processing events.

Use confirmed Stripe events to update payment and order state. Handle duplicate webhook delivery safely. Do not trust browser redirects for payment confirmation.

Keep the implementation limited to the events required by our current checkout flow.

---

## 30 Build Checkout UI With `/build-ui`

PROMPT

`/build-ui` Build the checkout experience around the payment flow we've implemented. Show the cart summary, quantities, totals, checkout action, loading states and useful failure or cancellation states.

---

## 31 Build Payment Success UI With `/build-ui`

PROMPT

`/build-ui` Build the post-checkout order confirmation experience. Use the application's confirmed order state. Show a pending state if payment confirmation hasn't completed yet.

SECTION

# ORDERS & CUSTOMER ACCOUNT

## 32 Add Order History With `/build-ui`

PROMPT

`/build-ui` Add order history to the authenticated customer account. Only show orders belonging to the signed-in customer. Include order date, payment status, total and access to individual order details.

---

## 33 Build Order Details With `/build-ui`

PROMPT

`/build-ui` Build the customer order detail experience. Show the order items, quantities, prices, total and current order state.

The server must only return an order if it belongs to the authenticated customer.

SECTION

# ADMIN

## 34 Plan the Admin Experience

Now that products, users and orders all exist:

```
/plan
```

PROMPT

Inspect the current ecommerce application and plan the admin experience.

Admins need to:

- view products
- create products
- edit products
- manage categories
- view current stock
- update stock
- view orders

Every admin route and server-side action must verify the user's admin role.

Reuse the existing database and application patterns. Keep the first version focused. Don't add dashboards, analytics, warehouses or complex role systems yet.

Show me the routes, components, actions and verification strategy.

## 35 Implement the Admin Foundation

PROMPT

Implement the approved admin foundation.

Create the protected admin layout, navigation and route structure. Every admin route and server-side action must verify the user's admin role.

Do not rely only on hiding admin links from normal users. Keep this step focused on the admin foundation.

---

## 36 Build Admin Product Management

PROMPT

Implement product management inside the admin experience.

Admins should be able to:

- view products
- create a product
- edit product details
- update category
- update price
- update product availability

Reuse the existing Drizzle schema and server-side patterns. Validate inputs server-side.

Because `/build-ui` is intentionally written for **customer-facing ecommerce UI**, don't force it onto internal admin workflows.

That's another useful lesson: a Skill should be used where its instructions actually apply.

LESSON 06

# Subagents & Parallel Work

Only now do you introduce subagents.

The app is large enough that separate investigations can genuinely benefit from isolated context.

## 37 Investigate Inventory With Subagents

PROMPT

Before implementing inventory management, investigate it using separate focused subagents.

Delegate:

- One subagent to inspect the current database schema and stock model.
- One subagent to inspect the admin product editing flow and reusable patterns.
- One subagent to inspect the current testing and verification setup.

Have each return:

- current behavior
- relevant files
- important risks
- recommendation

Don't implement anything yet. Combine their findings into one concise summary afterward.

This is a good use of subagents because the investigations are independent.

## 38 Plan Inventory Management

Once the research comes back:

```
/plan
```

PROMPT

Using the inventory investigation we just completed, plan the admin inventory management feature.

Admins should be able to view current stock and update it safely.

Reuse the current database and admin patterns.

Keep inventory simple and continue using the existing stock field unless there is a clear reason not to.

Show:

- files that need to change
- validation rules
- server-side update flow
- how customer-facing availability remains consistent
- how we'll verify the feature

Don't implement anything yet.

## 39 Implement Inventory Management

PROMPT

Implement the approved inventory management plan.

Allow admins to view current stock and update it safely. Validate stock values server-side. Make sure customer-facing availability continues to reflect the database correctly.

Do not add warehouses, reservations or inventory history that aren't part of the approved plan.

Verify the feature when you're done.

---

## 40 Use `/build-ui` for Customer Stock States

The internal admin workflow isn't a match for the Skill, but the customer-facing low-stock state is.

PROMPT

`/build-ui` Add the approved low-stock and out-of-stock states to the customer-facing product experience using the existing stock data. Don't change the database unless it's actually necessary.

## 41 Create a Fresh Feature Reviewer Subagent

Create:

`.claude/agents/feature-reviewer.md`

with:

FEATURE-REVIEWER.MD

```
---
name: feature-reviewer
description: Review a completed feature in a fresh context after implementation. Use
when a feature needs an independent check for bugs, regressions, unnecessary
changes, project-rule violations and missing verification.
tools: Read, Grep, Glob
---

Review the requested feature without modifying the code.

Inspect the relevant implementation and surrounding code.

Return:

## Summary
What changed.

## Critical
Problems that can break functionality, data or security.

## Major
Important regressions or architectural issues.

## Minor
Smaller issues worth fixing.

## Verification
What was checked and what still needs verification.

If something cannot be verified, say so clearly.
```

## 42 Review Inventory in Fresh Context

PROMPT

Use the feature-reviewer subagent to independently review the inventory management feature.

Focus on:

- stock updates
- database behavior
- admin authorization
- customer-facing availability
- regressions
- missing verification

Don't make changes yet. Return the review first.

---

## 43 Review Checkout in Fresh Context

The same reviewer is particularly useful for payments.

PROMPT

Use the feature-reviewer subagent to independently review the checkout and payment implementation.

Focus on:

- trusted pricing
- stock validation
- Stripe webhook handling
- payment state
- order ownership
- authorization
- missing verification

Don't make changes yet. Return the review first.

## 44 Verify a Reported Problem

A subagent can still produce a false positive.

If a finding is uncertain:

PROMPT

Delegate this reported issue to a fresh subagent.

Verify whether the problem is actually reproducible before changing code.

Return:

- whether it is reproducible
- exact reproduction steps
- relevant files
- likely cause

Do not fix anything yet.

---

## 45 Fix Only Confirmed Problems

PROMPT

Fix only the confirmed issues from the review.

Do not redesign the feature or expand its scope. Run the relevant verification again after the fixes and summarize exactly what changed.

SECTION

# FINAL REVIEW

## 46 Run CodeRabbit Deep Scan

Once your important flows work, use CodeRabbit as another independent review layer. Run **CodeRabbit AI Deep Scan** against the project changes.

Then give Claude the findings:

PROMPT

Review these CodeRabbit findings against the actual implementation.

For each important finding, determine whether it is genuinely relevant and reproducible.

Group the results into:

- confirmed issues
- findings that need further verification
- false positives or irrelevant findings

Don't change code yet.

Then fix the confirmed issues rather than blindly accepting every suggestion.

## 47 Review the Whole Application

PROMPT

Review the ecommerce application as a whole.

Inspect:

- storefront
- product data flow
- authentication
- authorization
- account routes
- cart
- stock handling
- checkout
- Stripe payment flow
- webhook verification
- order ownership
- admin protection
- product management
- inventory management

Look for:

- broken functionality
- inconsistent data flow
- unnecessary duplication
- missing server-side validation
- authorization mistakes
- client data being trusted incorrectly
- payment issues
- stale project conventions
- missing verification

Don't add new features. Return the findings first.

## 48 Verify the Customer Flow

PROMPT

Verify the complete admin flow:

- sign in as admin
- access the admin area
- create or edit a product
- update stock
- confirm the storefront reflects that change
- inspect an order

Also verify that a normal user cannot access protected admin routes or actions.

Report anything that cannot be verified.

Don't add new features.

## 49 Verify the Admin Flow

PROMPT

Verify the complete customer flow:

- sign up or sign in
- browse products
- view a product
- add it to cart
- change quantity
- attempt an invalid quantity
- checkout
- complete payment
- confirm the order
- view the order from the account

Report anything that cannot be verified. Don't add new features.

## 50 Sync the Final Project Context

Finally:

PROMPT

Review CLAUDE.md against the current repository.

Update it with only the durable project decisions Claude should remember in future sessions.

Remove anything outdated. Don't add information Claude can easily discover from reading the repository.

If you've installed the Agentic Engineering workflow shown during the course, this is also where you can use:

```
/sync
```

# Complete Course Flow

COURSE-FLOW.TXT

```
LESSON 01
Project setup
→ Explore repository

LESSON 02
CLAUDE.md

LESSON 03
Design system
→ Homepage
→ Product listing
→ Product details

LESSON 04
Plan Mode
→ PostgreSQL
→ Drizzle
→ Real product data

LESSON 05
Create /build-ui
→ New Arrivals
→ Categories
→ Search
→ Better Auth Skill
→ Authentication
→ /build-ui auth screens
→ /build-ui account
→ Cart
→ /build-ui cart
→ Stripe plugin
→ Checkout
→ /build-ui checkout
→ /build-ui order history
```

COURSE-FLOW.TXT (CONTINUED)

```
ADMIN
Admin foundation
→ Product management

LESSON 06
Subagent research
→ Inventory
→ Fresh feature reviewer
→ Checkout review
→ Verify findings

FINAL
CodeRabbit Deep Scan
→ Full review
→ Customer verification
→ Admin verification
→ Sync project context
```

The important thing is that the project now grows **with the course**.

| | |
|---|---|
| 01 | You first learn how Claude works. |
| 02 | Then how to give it project memory. |
| 03 | Then how to control it while building UI. |
| 04 | Then how to plan expensive decisions. |
| 05 | Then how to turn repeated workflows into Skills. |

And only after that do you start using those Skills throughout the rest of the application.

Finally, once the application becomes large enough for delegation to actually be useful, you introduce subagents.

That way every Claude Code feature appears because the project created a real reason to use it.